Escuela de Ciencias de la Computación
Parcial 2026-II
CC4P1 Programación Concurrente y Distribuida

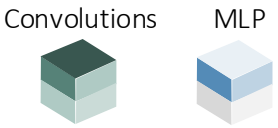
Open AI Cubes es un concepto arquitectónico para sistemas de Inteligencia Artificial modulares, donde cada "cubo" representa una capacidad de IA en una dimensión específica. Estos cubos se despliegan en entornos distribuidos (múltiples nodos) y cooperan en paralelo para procesar una red neuronal convolucional (CNN)

Usted deberá diseñar e implementar una versión funcional de Open AI Cubes que integre el tratamiento de imágenes. Todo el sistema debe construirse exclusivamente con el SDK escogido, sin librerías externas

Micro chunk

Un Micro chunk en una micro parte que es concurrentes paralelo y distribuido que será usado posteriormente por los módulos principales que se ejecutan en paralelo distribuido y concurrente.

- Convolutions
- MLP (Multilayer Perceptrons)



Una red neuronal convolucional (CNN) es una arquitectura de aprendizaje profundo diseñada para procesar datos con estructura de rejilla, como imágenes. Se compone de dos partes fundamentales: las convoluciones y el perceptrón multicapa (MLP).

La primera parte, las convoluciones, se encarga de extraer características de la entrada. Mediante filtros que se deslizan sobre la imagen, la red detecta patrones locales como bordes, texturas y formas, y a medida que avanza en profundidad, patrones más complejos como objetos completos. A esta etapa se le suman operaciones de activación (como ReLU) y de reducción dimensional (pooling), que hacen la representación más compacta y abstracta. El resultado final de esta parte es un vector de características que resume la información más relevante de la entrada.

La segunda parte, el MLP (Multilayer Perceptron), toma ese vector de características y lo procesa mediante capas totalmente conectadas para producir una predicción final, ya sea una clasificación, una regresión u otra tarea. Aquí las neuronas combinan globalmente la información extraída y, tras varias capas con funciones de activación, se obtiene la salida deseada (por ejemplo, mediante una función Softmax para clasificación).

En resumen, una CNN puede entenderse como un extractor de características convolucional seguido de un clasificador tipo perceptrón multicapa: las convoluciones aprenden qué hay en la imagen y el MLP decide a qué clase pertenece o qué valor le corresponde.

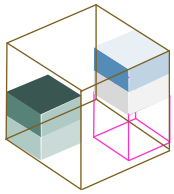
Módulo

Se deben implementar un módulo principal:

- Modelo de procesamiento de Imágenes

Cada módulo debe reutilizar los micro chunks que requiera para su consumo también de manera concurrentes paralela y distribuida.

Modelo de procesamiento de Imágenes

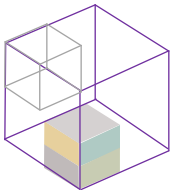


Extensión del Sistema

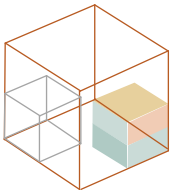
Desarrollar módulos adicionales complementarios para la aplicación:

- Backend: Desarrollar un módulo que consuma los resultados del modelo de Texto Imágenes, respondiendo a consultas de los clientes.
- Frontend: Implementar una interfaz de escritorio (desktop) que permita visualizar los resultados al usuario final.

Modulo Backend



Modulo Frontend

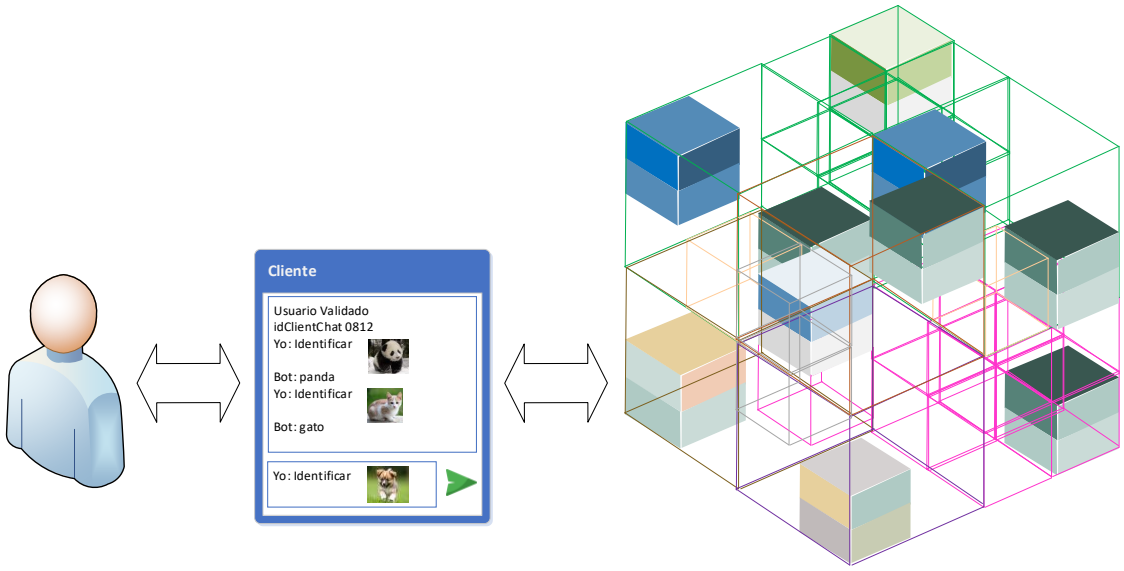


Modulo Frontend



Integración

Integrar todos los módulos (Texto, Imágenes, Backend, Frontend) en un sistema distribuido. Asegurar que el protocolo de comunicación entre nodos permita responder consultas de chat de los clientes de manera eficiente.



Comunicación entre Nodos

Diseñar el protocolo de comunicación entre nodos usando únicamente el SDK del lenguaje elegido (ejemplo: sockets, threads).

Evaluar en varios casos con los tiempos con estos casos.

- **Secuencial** (1 proceso [1 Hilo] secuencial)
- **Paralelo** (1 procesador con N núcleos físicos misma maquina con H hilos por N núcleos)
- **Distribuido** (M máquinas, 1 núcleo con 1 hilo por núcleo)
- **Paralelo Distribuido** (M máquinas con N núcleos físicos misma maquina con H hilos por N núcleos)

Donde:

LP = lenguaje de programación.

Desarrollo de los Cliente y el Servidor Central (LP1) y Nodos “n” (LP2), donde se tienen que cumplir con los siguientes casos:

- Caso 1 donde LP1 = LP2 (ejem se ejecute como mínimo jdk 8) (todo el sistema distribuido paralelo funciona con java).
- Caso 2 donde LP1 <> LP2, donde los(s) Nodo(s) está en otro lenguaje que no sea Java.
- Caso 3 los que tienen grupo de tres o más agregar otros lenguajes de programación en Nodo(s) adicional(es) (ejem LP3, LP4, etc.) se toma en cuenta la cantidad de lenguajes de programación.

Desarrollar en un cluster de datos para desplegar el programa.

Desarrollando el servidor en internet o en la nube (opcional).

Describir la arquitectura diseñada.

Describir el diagrama de protocolo.

No usar websocket, no usar socket io, no usar algún middleware, no usar otras librerías de conexión u otros protocolos ya trabajados por un tercero.

Usar las Apis, librerías o sdk estándares del lenguaje de programación base.

Explicar el Desarrollo del programa.

Subir en Univirtual.

Se consultará una pregunta en su cluster u otro.

Hacer pruebas donde los clientes puedan enviar cientos de transacciones regulando con un delay aleatorio entre las transacciones, para saturar el sistema y ver sus límites.

Hacer notas de recordatorio para su exposición.

Comprimido consta

- Subir códigos fuentes de en extensión en el LP1 y LP2 (o LP3), según sea el caso.
- PDF Informe.
- PDF Presentación.
- Solo subir las extensiones del lenguaje de programación por ejemplo java, bd, pesos, etc.., que sean necesarios para las ejecuciones, si suben otras extensiones no necesarias como historial del github se descontará puntaje.
- Evaluar el desempeño con el mayor número de nodos en red con graficas.

Parte 1